Java Singleton — Notes

What is required to make a class singleton?

- Private constructor — prevents external code from creating instances with new.
- Static factory method — the only way to get the instance (usually getInstance()).

1. Eager Instantiation

Instance is created when the class is loaded by the JVM (not when actually needed).

```
public class Service {
    private static final Service service = new Service();
    private Service() { }
    public static Service getInstance() {
        return service;
    }
}
```

- Advantage: simple, thread-safe automatically (JVM handles class loading safely).
- Drawback: memory wasted if the instance is never actually used.

2. Lazy Instantiation

Instance is created only when first requested.

```
public class Service {
    private static Service service;
    private Service() { }
    public static Service getInstance() {
        if (service == null) {
            service = new Service();
        }
        return service;
    }
}
```

- Advantage: instance created only when needed — saves memory.
- Drawback: not thread-safe — multiple threads calling getInstance() simultaneously can each see null and create separate instances, violating the singleton principle.

3. Thread-safe Instantiation

Same as lazy, but the factory method is synchronized so only one thread at a time can access it.

```
public class Service {
    private static Service service;
    private Service() { }
    public synchronized static Service getInstance() {
        if (service == null) {
            service = new Service();
        }
        return service;
    }
}
```

- Advantage: thread-safe and lazy.
- Drawback: synchronized locks the entire method on every call, even after the instance already exists — a performance issue under load.

4. Double-checked Locking

Only synchronizes during the first-time creation; skips the lock on every call afterward.

```
public class Service {
    private static volatile Service service;
    private Service() { }
    public static Service getInstance() {
        if (service == null) {
            synchronized (Service.class) {
                if (service == null) {
                    service = new Service();
                }
            }
        }
        return service;
    }
}
```

- Advantage: thread-safe, lazy, and good performance (lock happens only once).
- Drawback: complex to implement correctly.
- Important: the variable must be volatile — otherwise a thread could see a non-null reference to a half-constructed object due to instruction reordering.

5. Bill Pugh Instantiation

Uses a static inner holder class so the instance is created lazily and thread-safely, with no explicit locking needed.

```
public class Service {
    private Service() { }
    private static class Holder {
        private static final Service INSTANCE = new Service();
    }
    public static Service getInstance() {
        return Holder.INSTANCE;
    }
}
```

- Works because the JVM only loads the Holder class (and creates INSTANCE) the first time getInstance() is called — combining lazy loading with JVM class-loading thread safety.

Quick comparison

Approach	Lazy?	Thread-safe?	Performance	Complexity
Eager	No	Yes	Good	Simple
Lazy (basic)	Yes	No	Good	Simple
Thread-safe (synchronized method)	Yes	Yes	Poor (locks every call)	Simple
Double-checked locking	Yes	Yes	Good	Complex
Bill Pugh (holder class)	Yes	Yes	Best	Simple